RAG: the complete recap

What each method does, when to use it, its pros and cons, and how it connects to ILLUIN Technology's work. Prepared for the Junior Applied AI Engineer interview.

> **One-sentence definition.** Retrieval-Augmented Generation: before answering, the system retrieves relevant passages from your own data and gives them to the LLM with the question, so the answer is grounded in real documents instead of the model's memory.

1. The big picture

A RAG system works in two separate moments.

```
BEFORE ANY QUESTION (offline, done once)
  INDEXING: load docs -> split into chunks -> embed -> store in a vector DB
     +- Multi-representation : search with summaries, return full documents
     +- RAPTOR               : add a tree of summaries (for broad questions)
     +- ColBERT              : one vector per token (for precise matching)

AT QUESTION TIME (online)
  1. Query translation   -> rewrite the question so it searches better
  2. Routing             -> choose the source (docs, SQL, web) or the prompt
  3. Query construction  -> write SQL / Cypher / filters if the source is structured
  4. Retrieval           -> search the index
  5. Generation          -> the LLM answers from the retrieved pieces

  Corrective / Active RAG wraps steps 4-5: check, fix if needed, then answer.
```

Block	Question it answers	Acts on
Indexing	What can be found?	The stored data
Query translation	Is the question well phrased for search?	The question (text)
Routing	Where should we search?	The destination
Query construction	How do we query structured data?	The question (into SQL / filters)
Retrieval	Which pieces are relevant?	The index
Corrective RAG	Are the results good enough?	The flow (checks + loops)
Generation	What is the final answer?	The LLM output

2. Indexing (basic)

Purpose: prepare the documents so they can be searched. If indexing is bad, no query trick can save retrieval.

1. **Load** documents (PDF, web, Office) and extract text.
2. **Chunk**: split into pieces (e.g. 300-1000 tokens, with ~10% overlap).
3. **Embed**: turn each chunk into a vector (e.g. sentence-transformers, OpenAI embeddings).
4. **Store** in a vector DB (FAISS, Chroma, Qdrant, Weaviate, pgvector).

Choice	Use / context	Pros	Cons
Small chunks	Precise fact lookups	**Sharp, precise match**	**Lose surrounding context**
Big chunks	Explanations, reasoning	**Keep context**	**Fuzzy vector (many ideas mixed), more tokens to the LLM**
Fixed-size split	Quick baseline	**Simple, fast**	**Cuts sentences and ideas**
Recursive split	Default for most text	**Respects paragraphs, then sentences**	**Still blind to topic changes**
Semantic split	Long, mixed-topic documents	**Cuts where the topic changes**	**Slower (needs embeddings to split)**
Multi-representation (parent document)	When you need both precise search and full context	**Search with a small summary/chunk, return the full parent document: best of both**	**Extra storage; summaries cost LLM calls**

3. Query translation

Purpose: users ask badly (vague, too specific, wrong vocabulary, several questions at once). Rewrite the question before searching. **Applies to text search** (vector DB).

Method	How it works	When to use	Pros	Cons
Multi-query	LLM writes 3-5 rephrasings; search with each; merge results	Vague questions, varied vocabulary	**More chances to hit the right doc**	**N searches + 1 LLM call; more noise**
RAG Fusion	Multi-query + Reciprocal Rank Fusion: score(doc) = Σ 1/(k + rank), k≈60; keep top docs	Same as multi-query, when ranking quality matters	**Docs found by several queries rise to the top; cleaner ranking**	**Same cost as multi-query**
Decomposition	Split into sub-questions; answer each (sequentially or in parallel); combine	Complex, multi-part or comparison questions	**Handles multi-hop reasoning**	**Many LLM calls; errors can propagate**
Step-back	Also ask a more general question; retrieve for both	Very specific questions over docs with general rules/concepts	**Retrieves principles and background**	**Useless for direct fact lookups; adds noise**
HyDE	LLM writes a fake answer; embed that and search with it	Short questions, vocabulary far from the documents	**Answer-like text matches documents better**	**Fails if the LLM knows nothing about the domain; +1 LLM call**

Memory hook: multi-query / RAG Fusion = same level, more versions; decomposition = smaller pieces; step-back = more general; HyDE = change the form (answer instead of question).
Key point: the LLM never sees rankings. Ranking only decides *which* chunks enter the prompt and in what order (best first: LLMs attend less to the middle of long contexts, "lost in the middle").

4. Routing

Purpose: choose where to send the question (a document base, a SQL database, the web) or which prompt to use, before searching.

Method	How it works	When to use	Pros	Cons
Logical routing	An LLM picks a route from a list, as structured output (JSON / enum)	Routes that need understanding to tell apart	**Flexible, handles nuance**	**+1 LLM call per question; can misroute**
Semantic routing	Embed route descriptions once; send the question to the most similar one	Clearly distinct routes (e.g. physics vs math prompt)	**Fast, cheap, no LLM call**	**Less precise when routes overlap**

Tip: always keep a default route ("general search" or "I don't know") for questions that match nothing.

5. Query construction

Purpose: similarity search cannot apply exact conditions ("after 2024") or compute anything ("top 3", "total"). The LLM translates the question into the language of a structured source.

Method	Source	Example	Pros	Cons
Self-query (metadata filters)	Vector DB with metadata	"RAG videos after 2024" → search "RAG" + filter year > 2024	**Combines meaning and exact filters**	**Needs clean metadata and a declared schema**
Text-to-SQL	Relational DB	"Top 3 clients in 2025" → SELECT ... GROUP BY ... LIMIT 3	**Exact answers, aggregations**	**Wrong SQL risk; needs the schema; must be read-only and validated**
Text-to-Cypher	Graph DB (Neo4j)	"Colleagues of X on project Y" → path query	**Multi-hop relationships**	**Hard to generate correctly; graph must exist**

> **Translation vs construction:** text only → query translation. Tables / graphs → query construction. Text + metadata → both (semantic part improved by translation, filter written by construction). Routing decides which case applies.

6. Advanced indexing: RAPTOR and ColBERT

Method	How it works	When to use	Pros	Cons
RAPTOR	Cluster chunks (embeddings, UMAP + Gaussian mixture), summarize each cluster with an LLM, repeat upward → tree of summaries. Store all levels together; normal similarity search picks the right level.	Long documents / large collections with overview or synthesis questions	**Answers both detail and "big picture" questions; retrieval unchanged**	**Many LLM calls to build; rebuild when docs change**
ColBERT	One vector per token (doc and query). Score = for each query token, best similarity with any doc token (MaxSim), summed: "late interaction".	Precision matters: names, technical terms, specific wording; also as a reranker	**Much finer matching than one vector per chunk**	**Far more storage; heavier compute**

RAPTOR changes only indexing (what is stored). ColBERT changes both indexing (many vectors per chunk) and retrieval (MaxSim scoring).

7. Retrieval and generation

Technique	Use	Pros	Cons
Dense retrieval	Default: cosine similarity on embeddings	**Captures meaning, synonyms**	**Misses exact codes, names, rare terms**
Hybrid (BM25 + dense)	Enterprise docs with product codes, names	**Keywords + meaning**	**Two indexes; score fusion to tune**
Reranking (cross-encoder)	Re-sort the top 20-50 to keep the best 3-5	**Big precision gain**	**Added latency**
Grounded prompt + citations	"Answer only from these extracts [1]..[k]"	**Fewer hallucinations, traceable**	**Model may still drift; context length limits**

8. Corrective RAG (Active RAG) and the state machine

Purpose: classic RAG runs once and never checks itself. Active RAG adds checkpoints and loops.

```
retrieve -> grade documents -> relevant?
              yes -> generate
              no  -> transform query -> web search -> generate
(optional) after generate: is the answer supported by the docs? if not -> regenerate (max N
tries)
```

Step	What it does
Retrieve	Normal search in the index
Grade documents	An LLM judges each chunk relevant / not; bad ones are dropped
Transform query	Rewrite the question for a web search engine
Web search	Fallback source when the index lacks the answer (e.g. Tavily)
Generate	Answer only from the kept documents

The graph is the flowchart of steps, not a data graph: *nodes* = steps (Python functions), *edges* = arrows, *conditional edges* = decisions, *state* = shared dict (question, documents, answer). Built with **LangGraph**; unlike a chain, it can branch and loop.

Use when retrieval often misses; errors are costly (legal, banking, health, support); several sources available as fallback.

Avoid when simple RAG already works: every check is an extra LLM call (latency, cost).

Variants: Self-RAG (the model decides whether to retrieve and critiques itself), Adaptive RAG (a router picks no retrieval / simple RAG / multi-step depending on difficulty).

9. Evaluating a RAG system

Stage	Metric	What it tells you
Retrieval	Recall@k, MRR, nDCG	Is the right passage in the top k, and how high?
Generation	Faithfulness, answer relevance (Ragas, LLM-as-judge)	Is the answer supported by the passages, and does it answer the question?
System	Latency, cost per query, human review	Is it usable in production?

Always evaluate retrieval and generation separately: a bad answer can come from bad retrieval or from bad generation, and the fixes are different.

10. Link with ILLUIN Technology

Based on public information (company page, CentraleSupélec / ANR announcements, the job offer, published research). It describes their known focus areas, not their internal stack.

What ILLUIN does

- ~170 people, founded 2017, Puteaux (La Défense); 100+ clients. Custom projects in **agentic, generative, conversational and document AI**, plus their own products: conversational AI, intelligent document processing (parsing, search), customer voice analysis.
- **LIAGORA**: joint lab with CentraleSupélec (MICS), supported by ANR, on (1) design and orchestration of LLM agents, (2) evaluation and design of **textual and multimodal RAG**, (3) securing assistants with hybrid rules + AI engines.
- Research on document retrieval: the team behind **ColPali** (Faysse et al., 2024), which retrieves document pages directly from their images with a vision-language model.
- The Junior Applied AI Engineer role: from POC to **industrial-scale multi-agent systems**, client relationship, state-of-the-art watch.

How each RAG block maps to their work

RAG block	Where it shows up at ILLUIN	What to say in the interview
Indexing / parsing	Document AI: parsing and search over client documents (PDFs, scans, tables)	Chunking trade-offs; why parsing quality bounds RAG quality; multi-representation for long documents
ColBERT	**ColPali** = ColBERT's late interaction (MaxSim) applied to page-image patches embedded by a VLM: no OCR pipeline needed	"ColBERT gives one vector per token and scores with MaxSim; ColPali does the same on image patches, which handles tables, charts and layout that text extraction loses"
RAPTOR / multi-representation	Long reports, regulatory and contractual corpora where users ask synthesis questions	When overview questions fail with flat chunks, add summary levels
Query translation	Conversational assistants: users ask short, vague or multi-part questions	Pick per failure mode: HyDE for vocabulary gaps, decomposition for multi-part, step-back for policy-type questions
Routing + query construction	Assistants that mix documents with business databases and APIs	Route to docs vs SQL vs tools; text-to-SQL with schema in the prompt and read-only validation
Corrective / Active RAG	LIAGORA's agent orchestration and the multi-agent systems in the job offer	LangGraph state machine: grade, rewrite, fallback, hallucination check; cap the loops to control cost
Evaluation	LIAGORA axis on RAG evaluation; public benchmarks released with their retrieval research	Separate retrieval metrics (recall@k) from generation metrics (faithfulness); build a small annotated test set first
Guardrails	LIAGORA axis on securing assistants (hybrid rules + AI)	Deterministic rules for business constraints + LLM checks; refuse when context is missing

Your bridge from CAST

- Embeddings + cosine similarity = the retrieval core of RAG (you built it: all-MiniLM, semantic search).
- Dual-LLM agreement + expert review of low-confidence cases = the same logic as Corrective RAG's grading step and human-in-the-loop guardrails.
- Benchmarking precision / recall / latency / inference cost = RAG evaluation and production trade-offs.
- UMAP + HDBSCAN clustering of packages = the clustering step inside RAPTOR.

Sources: ILLUIN page (Welcome to the Jungle); Mesinfos, CentraleSupélec x ILLUIN (LIAGORA); ILLUIN job offer; Faysse et al., ColPali (2024); LangChain "RAG from scratch".